

Тестове завдання для Laravel Developer

1. Завдання

Потрібно розробити REST API на Laravel, що:

1. Асинхронно імпортує пропозиції житла від постачальників.
2. Повертає найдешевшу актуальну пропозицію для кожного об'єкта.
3. Дозволяє безпечно забронювати пропозицію.

Це не завдання на frontend. Нас цікавлять структура бази даних, робота з Laravel, SQL-запити, черги, транзакції та автоматизовані тести.

2. Технології

- PHP 8.2+
- Laravel 11 або 12
- MySQL 8+
- черга через mysql або Redis.

Docker дозволений, але не обов'язковий.

3. Основні сутності

- **Supplier** — постачальник пропозицій;
- **Property** — квартира або інший об'єкт житла;
- **Offer** — пропозиція постачальника для певних дат;
- **Import** — факт імпорту даних;
- **Reservation** — бронювання пропозиції.

Структуру таблиць, зв'язки, індекси та назви моделей кандидат проектує самостійно.

Потрібно створити seeder для двох постачальників: supplier-a, supplier-b

4. Імпорт пропозицій

Endpoint

POST /api/imports

Content-Type: application/json

Приклад запиту:

```
{
  "supplier": "supplier-a",
  "external_import_id": "import-2026-09-01-001",
  "sent_at": "2026-09-01T10:00:00Z",
  "offers": [
    {
      "external_id": "offer-a-10001",
      "property": {
        "code": "BCN-0001",
        "name": "Apartment near Sagrada Familia",
        "city": "Barcelona"
      },
      "check_in": "2026-10-10",
      "check_out": "2026-10-15",
      "max_guests": 4,
      "price": 72500,
      "currency": "EUR",
      "available_units": 2,
      "expires_at": "2026-09-10T23:59:59Z"
    }
  ]
}
```

Endpoint повинен:

1. Перевірити структуру запиту та наявність постачальника.
2. Створити запис імпорту.
3. Поставити обробку імпорту в чергу.
4. Одразу повернути 202

Приклад відповіді:

```
{
  "data": {
    "id": 15,
    "status": "pending"
  }
}
```

Правила імпорту

- Комбінація `supplier` + `external_import_id` повинна бути унікальною.
- Повторне надсилання того самого імпорту не повинно створювати дублікати або повторно запускати обробку.
- Комбінація `supplier` + `offer.external_id` також унікальна.
- Якщо пропозиція з таким `external_id` вже існує в іншому імпорті, її дані потрібно оновити.
- Поле `sent_at` показує, коли постачальник сформував імпорт.
- Об'єкт житла потрібно знайти або створити за `property.code`.
- Після успішної обробки імпорт отримує статус `completed`, після помилки — `failed`.
- Обробка пропозицій повинна виконуватися всередині `Job`, а не в HTTP-запиті.

Допустимі статуси імпорту: `pending`, `processing`, `completed`, `failed`.

5. Перегляд статусу імпорту

Endpoint

GET `/api/imports/{import}`

Endpoint повинен кинути поточний стан асинхронного імпорту.

Приклад відповіді:

```
{
  "data": {
    "id": 15,
    "supplier": "supplier-a",
    "external_import_id": "import-2026-09-01-001",
    "sent_at": "2026-09-01T10:00:00Z",
    "status": "completed",
    "total_offers": 20,
    "processed_offers": 20,
    "error": null,
    "created_at": "2026-09-01T10:00:02Z",
    "completed_at": "2026-09-01T10:00:04Z"
  }
}
```

Поля `total_offers`, `processed_offers`, `error` і `completed_at` можна реалізувати у зручному для вас вигляді.

6. Пошук житла

Endpoint

GET /api/properties

Правила вибірки

Пропозиція вважається актуальною, якщо:

- дати збігаються з параметрами пошуку;
- `max_guests >= guests`;
- `available_units > 0`;
- `expires_at > now()`;
- місто збігається, якщо передано `city`.

Пошук найдешевшої пропозиції, сортування та пагінація повинні виконуватися на рівні бази даних. Не можна отримати всі пропозиції та групувати їх PHP колекціями.

Приклад запиту:

GET

/api/properties?city=Barcelona&check_in=2026-10-10&check_out=2026-10-15&guests=2&page=1

Скорочений приклад відповіді:

```
{
  "data": [
    {
      "code": "BCN-0001",
      "name": "Apartment near Sagrada Familia",
      "city": "Barcelona",
      "best_offer": {
        "id": 125,
        "supplier": "supplier-a",
        "price": 72500,
        "currency": "EUR",
        "available_units": 2,
        "expires_at": "2026-09-10T23:59:59Z"
      }
    }
  ]
}
```

Також повинен містити `next`, `prev`, `per_page`

7. Бронювання

Endpoint

POST /api/offers/{offer}/reservations

Content-Type: application/json

Приклад запиту:

```
{
  "client_reference": "web-order-9f782b1c",
  "customer_name": "John Smith",
  "customer_email": "john@example.com"
}
```

Успішна відповідь повинна мати статус 201 Created і містити створене бронювання.

У README потрібно пояснити, яким механізмом рішення захищене від двох одночасних бронювань останньої одиниці. Окремий повноцінний тест із паралельними процесами не обов'язковий.

8. Очікування щодо коду

Потрібно використати стандартні можливості Laravel там, де вони доречні:

- migrations і foreign keys;
- Eloquent models та relationships;
- Form Requests;
- API Resources;
- Jobs і queues;
- database transactions;
- factories і seeders;
- feature-тести.

Контролери не повинні містити всю бізнес-логіку. Водночас не потрібно штучно будувати DDD, Clean Architecture або велику систему абстракцій.

9. README

Надати посилання на Git-репозиторій з історією комітів.

Також має містити:

- інструкцію встановлення та запуску;
- команди запуску міграцій, seeders, queue worker і тестів;
- коротке пояснення ідемпотентності імпорту;
- Також потрібен `.env.example` без секретів.

10. Орієнтовний час

Орієнтований час виконання - **4 години**